Lesson-End Project
Building a Smart Appointment Scheduler Using JSON Server and JavaScript

Project agenda: To design and implement a real-time Smart Appointment Scheduler using HTML, CSS, JavaScript, and JSON Server, enabling users to book, reschedule, and cancel appointments dynamically while ensuring real-time updates, preventing double bookings, and improving communication between customers and service providers

Description: This project involves developing a real-time Smart Appointment Scheduler to streamline booking management for service-based businesses. The process includes setting up a mock backend using JSON Server, creating a responsive user interface, and implementing JavaScript logic to handle appointment booking, rescheduling, and cancellation. The system ensures dynamic updates and prevents double bookings. Tools such as VS Code, HTML, CSS, JavaScript, and JSON Server will be used to build and test the application.

Tools required: Visual Studio Code, JSON Server

Prerequisites: Basic knowledge of HTML, CSS, and JavaScript

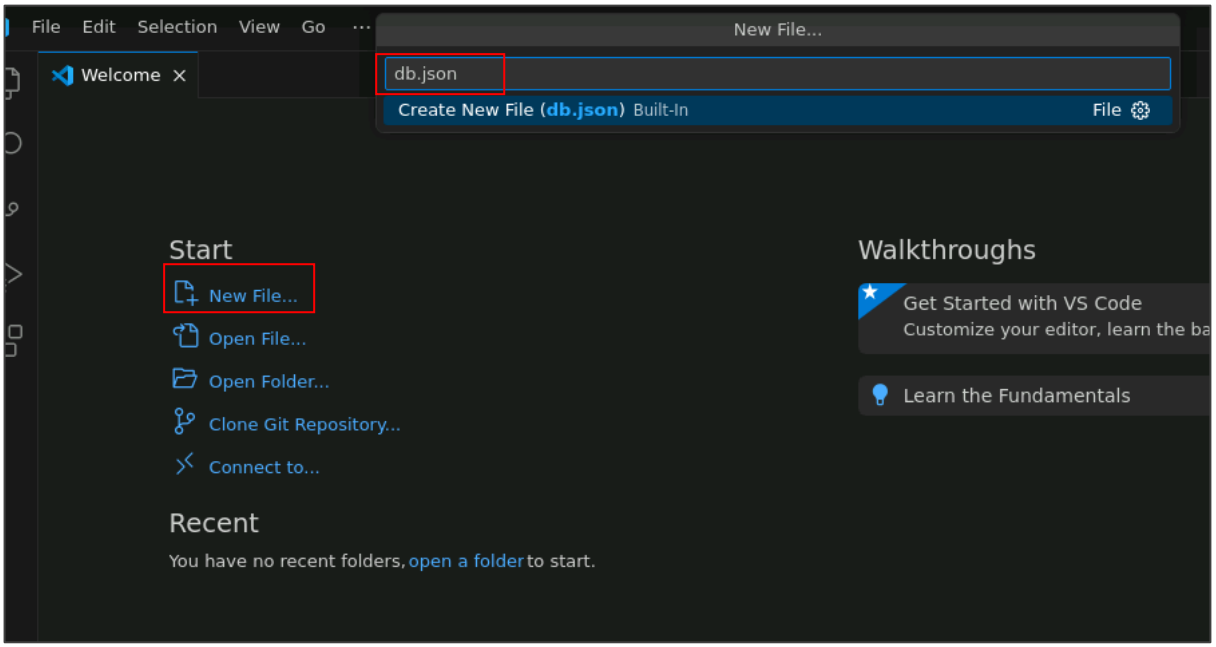
Expected deliverables: The final output will include a fully responsive HTML interface, a mock backend powered by JSON Server, and JavaScript logic for dynamic appointment management. The deliverables will demonstrate real-time updates, user interaction handling, and basic CRUD operations, providing a foundation for scalable and efficient appointment systems.

Steps to be followed:

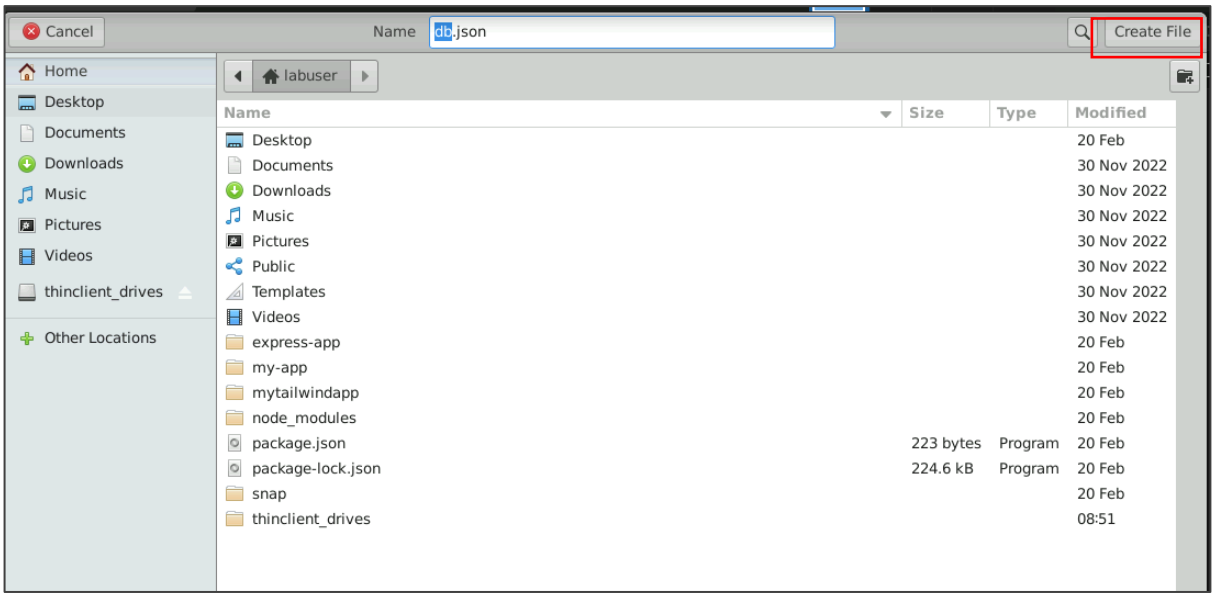
1. Set up the JSON server
2. Install and run json-server
3. Build the user interface
4. Implement appointment logic in JavaScript
5. Test and run the application

Step 1: Set up the JSON server

1.1 Open Visual Studio Code and create a **New File...** named **db.json**

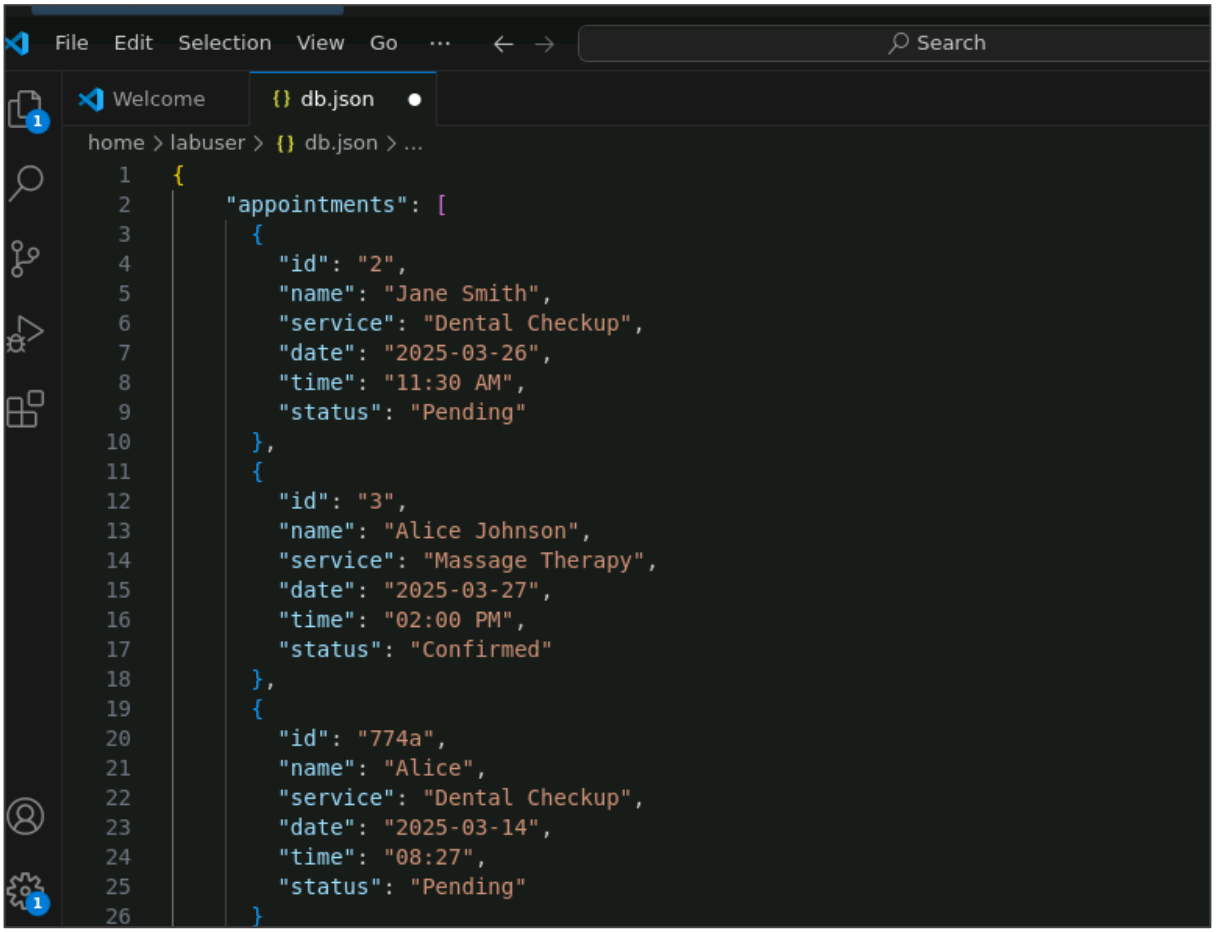


1.2 Click **Create File**



1.3 Copy and paste the following JSON structure into the **db.json** file:

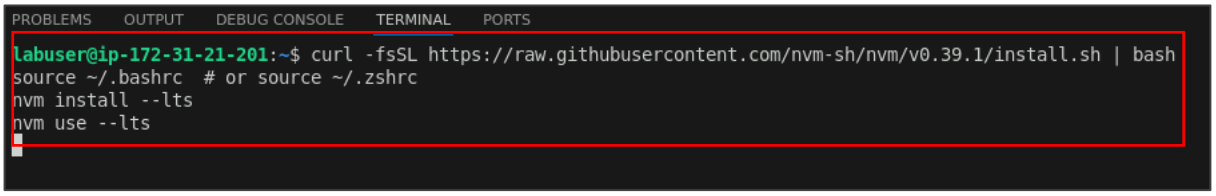
```
{
  "appointments": [
    {
      "id": "2",
      "name": "Jane Smith",
      "service": "Dental Checkup",
      "date": "2025-03-26",
      "time": "11:30 AM",
      "status": "Pending"
    },
    {
      "id": "3",
      "name": "Alice Johnson",
      "service": "Massage Therapy",
      "date": "2025-03-27",
      "time": "02:00 PM",
      "status": "Confirmed"
    },
    {
      "id": "774a",
      "name": "Alice",
      "service": "Dental Checkup",
      "date": "2025-03-14",
      "time": "08:27",
      "status": "Pending"
    }
  ]
}
```



Step 2: Install and run json-server

2.1 Open the terminal and run the following command to install nvm (Node Version Manager) and the latest LTS version of Node.js:

curl -fsSL https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.1/install.sh | bash
source ~/.bashrc # or source ~/.zshrc
nvm install --lts
nvm use --lts



2.2 Run the following command to install json-server:
npm install -g json-server

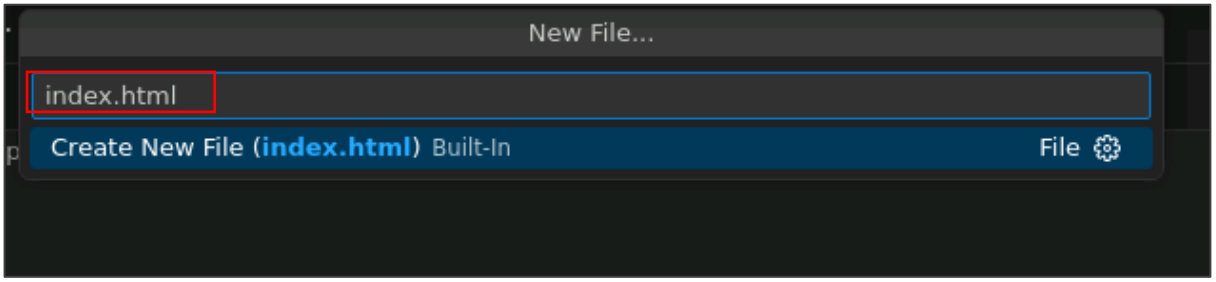
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS
labuser@ip-172-31-21-201:~$ npm install -g json-server
```

2.3 Run the following command to serve the JSON data:
json-server db.json --port 3000

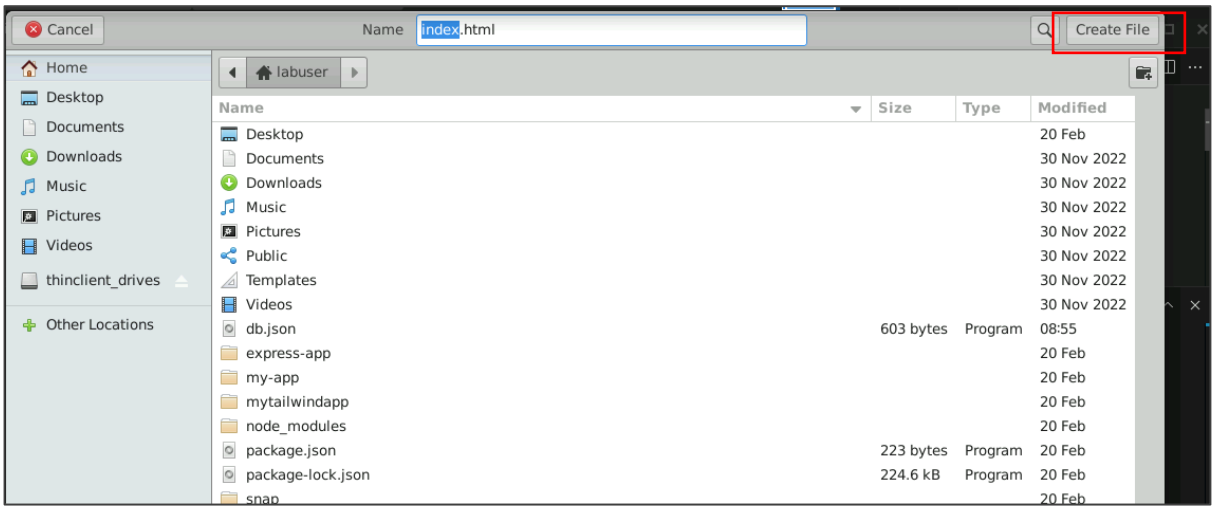
```
labuser@ip-172-31-21-201:~$ json-server db.json --port 3000
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching db.json...

♡( ◡‿◡ )

Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/appointments
```

Step 3: Build the user interface

3.1 Create an HTML file named **index.html**



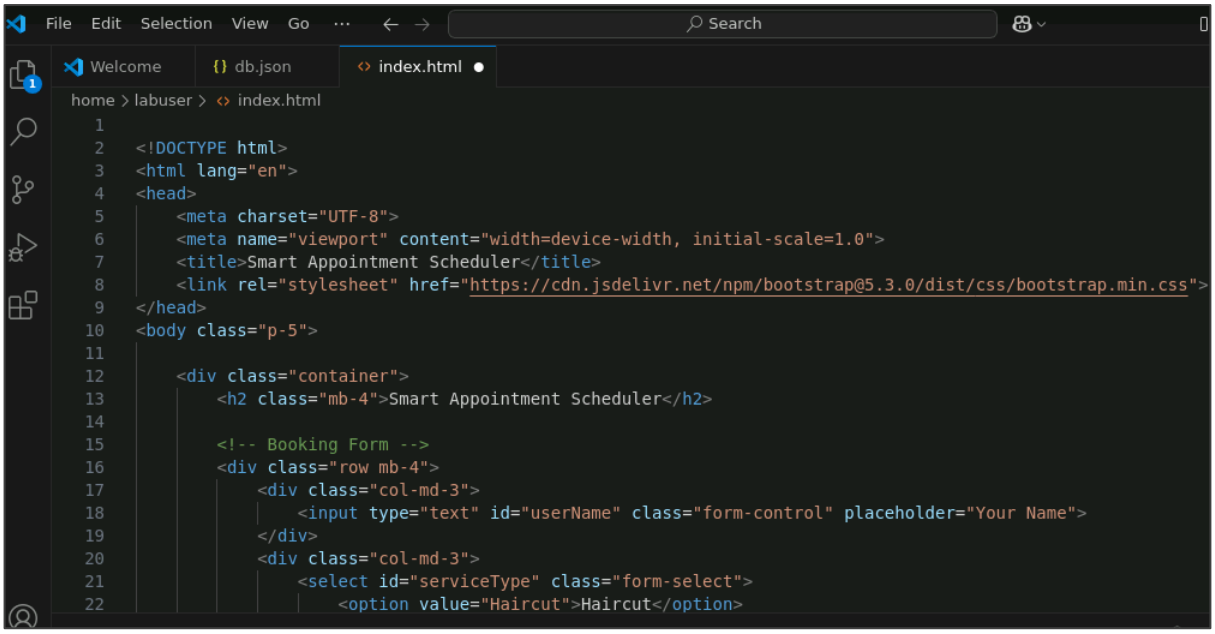
3.2 Click **Create File**



3.3 Copy and paste the following script to the **index.html** file:

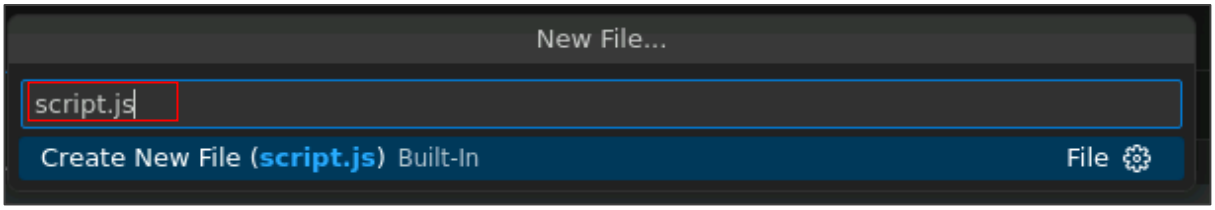
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Smart Appointment Scheduler</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css
">
</head>
<body class="p-5">

  <div class="container">
    <h2 class="mb-4">Smart Appointment Scheduler</h2>

    <!-- Booking Form -->
    <div class="row mb-4">
      <div class="col-md-3">
        <input type="text" id="userName" class="form-control"
placeholder="Your Name">
      </div>
      <div class="col-md-3">
        <select id="serviceType" class="form-select">
          <option value="Haircut">Haircut</option>
          <option value="Dental Checkup">Dental Checkup</option>
          <option value="Massage Therapy">Massage Therapy</option>
        </select>
      </div>
      <div class="col-md-3">
```

```
            <input type="date" id="appointmentDate" class="form-control">
          </div>
          <div class="col-md-2">
            <input type="time" id="appointmentTime" class="form-control">
          </div>
          <div class="col-md-1">
            <button class="btn btn-primary"
onclick="bookAppointment()">Book</button>
          </div>
        </div>

        <!-- Appointment List -->
        <h4>Upcoming Appointments</h4>
        <table class="table table-bordered">
          <thead>
            <tr>
              <th>Name</th>
              <th>Service</th>
              <th>Date</th>
              <th>Time</th>
              <th>Status</th>
              <th>Actions</th>
            </tr>
          </thead>
          <tbody id="appointmentsList"></tbody>
        </table>
      </div>

      <script src="script.js"></script>
    </body>
    </html>
```

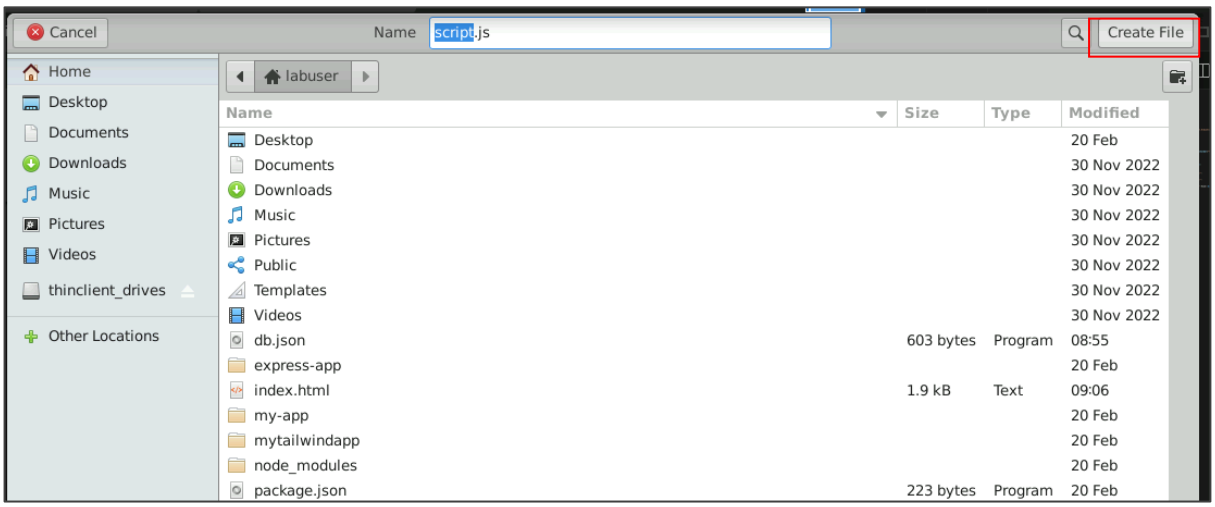


Step 4: Implement appointment logic in JavaScript

4.1 Create a script file named **script.js**



4.2 Click **Create File**



4.3 Copy and paste the following script into the script.js file to link it to the HTML file:

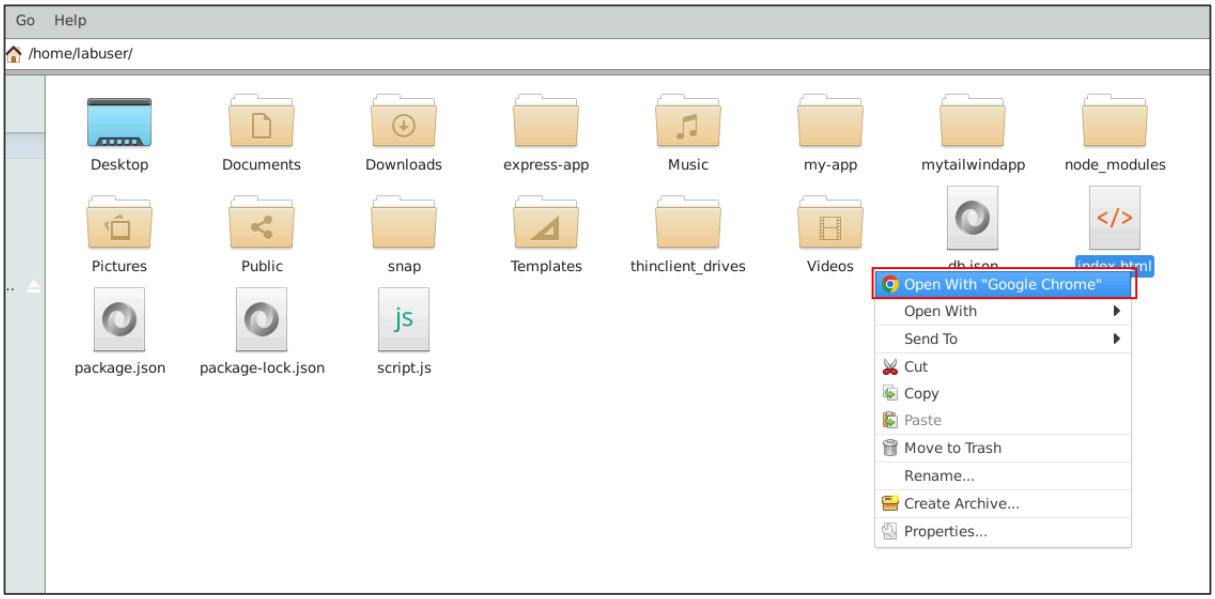
```
const API_URL = "http://localhost:3000/appointments";

// Fetch and display appointments
async function fetchAppointments() {
  const res = await fetch(API_URL);
  const appointments = await res.json();

  const tableBody = document.getElementById("appointmentsList");
  tableBody.innerHTML = "";

  appointments.forEach(appointment => {
    const row = document.createElement("tr");
    row.innerHTML = `
      <td>${appointment.name}</td>
      <td>${appointment.service}</td>
      <td>${appointment.date}</td>
      <td>${appointment.time}</td>
      <td>
        <span class="badge ${appointment.status === 'Confirmed' ? 'bg-success'
: 'bg-warning'}">
          ${appointment.status}
        </span>
      </td>
      <td>
        <button class="btn btn-sm btn-warning"
onclick="rescheduleAppointment(${appointment.id})">Reschedule</button>
        <button class="btn btn-sm btn-danger"
onclick="cancelAppointment(${appointment.id})">Cancel</button>
      </td>
    `;
    tableBody.appendChild(row);
  });
}

// Book a new appointment
async function bookAppointment() {
  const name = document.getElementById("userName").value;
  const service = document.getElementById("serviceType").value;
  const date = document.getElementById("appointmentDate").value;
  const time = document.getElementById("appointmentTime").value;

  if (!name || !date || !time) {
    alert("Please fill in all fields!");
    return;
  }
```

```
    const newAppointment = { name, service, date, time, status: "Pending" };

    await fetch(API_URL, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(newAppointment)
    });

    fetchAppointments();
}

// Reschedule appointment
async function rescheduleAppointment(id) {
    const newDate = prompt("Enter new date (YYYY-MM-DD):");
    const newTime = prompt("Enter new time (HH:MM AM/PM):");

    if (!newDate || !newTime) {
        alert("Invalid input!");
        return;
    }

    await fetch(`${API_URL}/${id}`, {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ date: newDate, time: newTime })
    });

    fetchAppointments();
}

// Cancel appointment
async function cancelAppointment(id) {
    await fetch(`${API_URL}/${id}`, { method: "DELETE" });
    fetchAppointments();
}

// Real-time polling every 3 seconds
setInterval(fetchAppointments, 3000);

// Initial load
fetchAppointments();
```

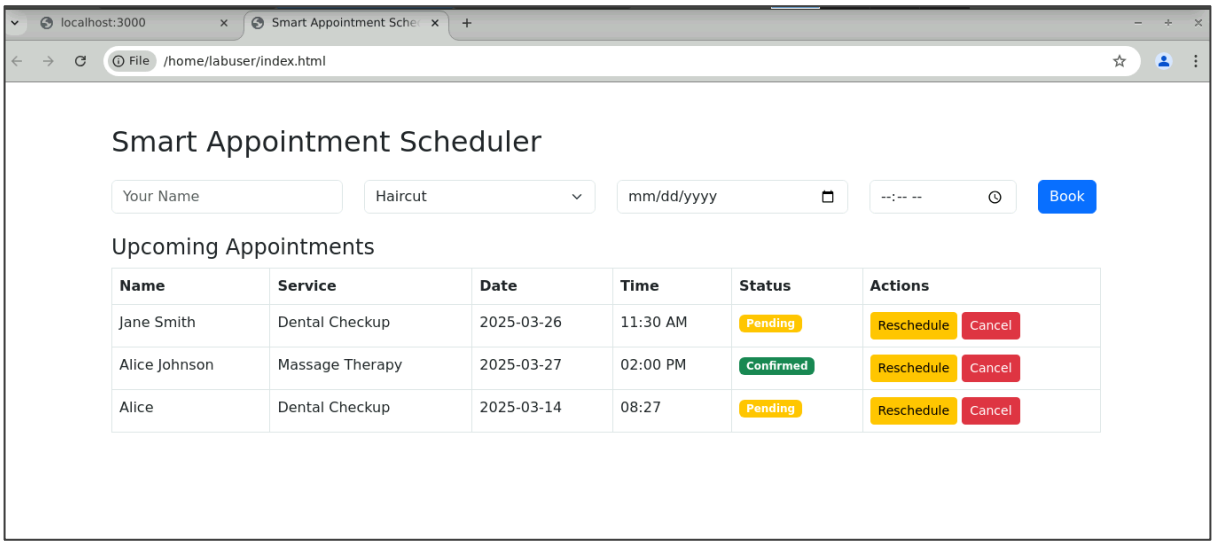
```
const API_URL = "http://localhost:3000/appointments";

// Fetch and display appointments
async function fetchAppointments() {
    const res = await fetch(API_URL);
    const appointments = await res.json();

    const tableBody = document.getElementById("appointmentsList");
    tableBody.innerHTML = "";

    appointments.forEach(appointment => {
        const row = document.createElement("tr");
        row.innerHTML = `
            <td>${appointment.name}</td>
            <td>${appointment.service}</td>
            <td>${appointment.date}</td>
            <td>${appointment.time}</td>
            <td>
                <span class="badge ${appointment.status === 'Confirmed' ? 'bg-success' : 'bg-warning'}">
                    ${appointment.status}
                </span>
            </td>
            <td>
                <button class="btn btn-sm btn-warning" onclick="rescheduleAppointment(${appointment.id})">Reschedule</button>
                <button class="btn btn-sm btn-danger" onclick="cancelAppointment(${appointment.id})">Cancel</button>
            </td>
```

Step 5: Test and run the application

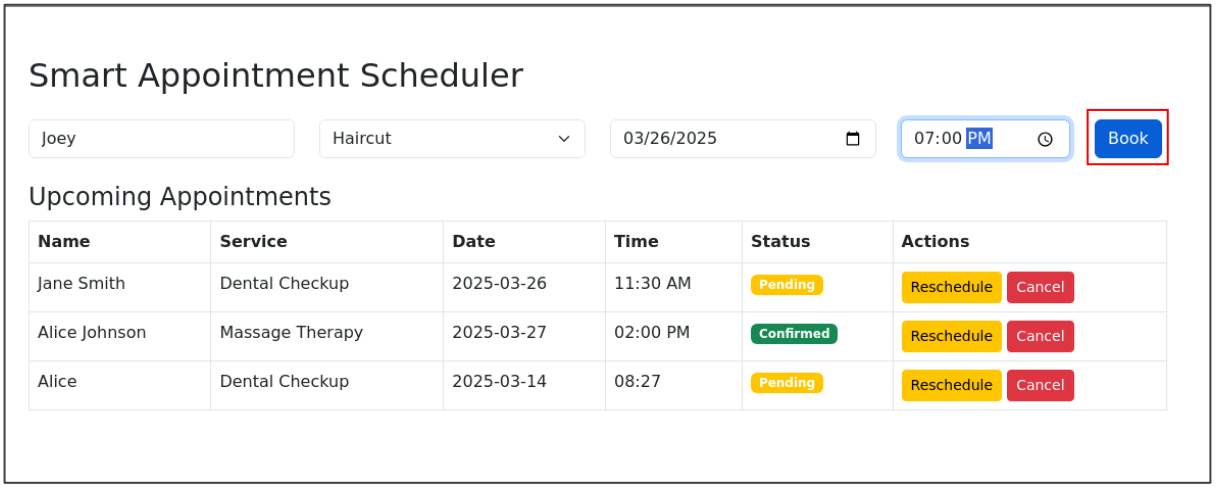
5.1 **Right-click** the index.html file and select **Open With "Google Chrome"** to view it in the browser



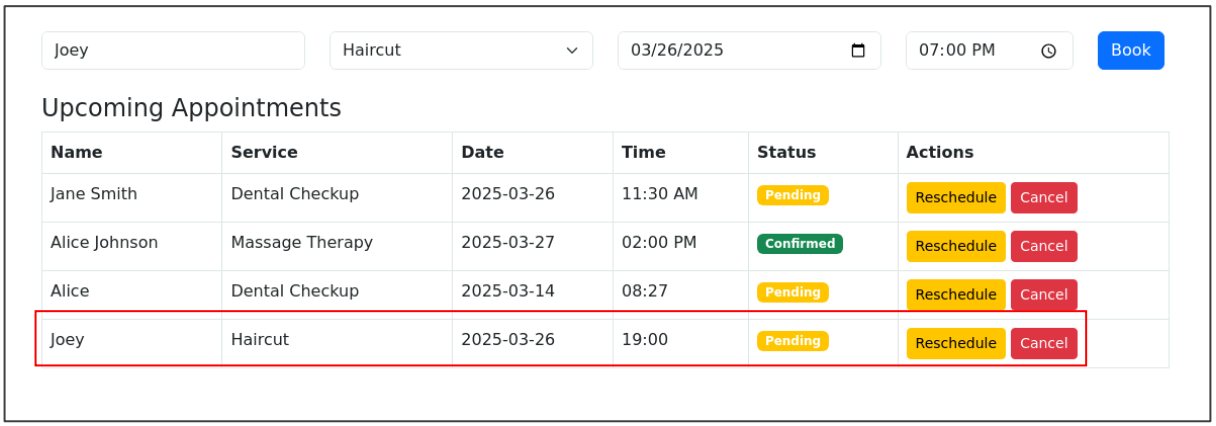
The output is as shown below:



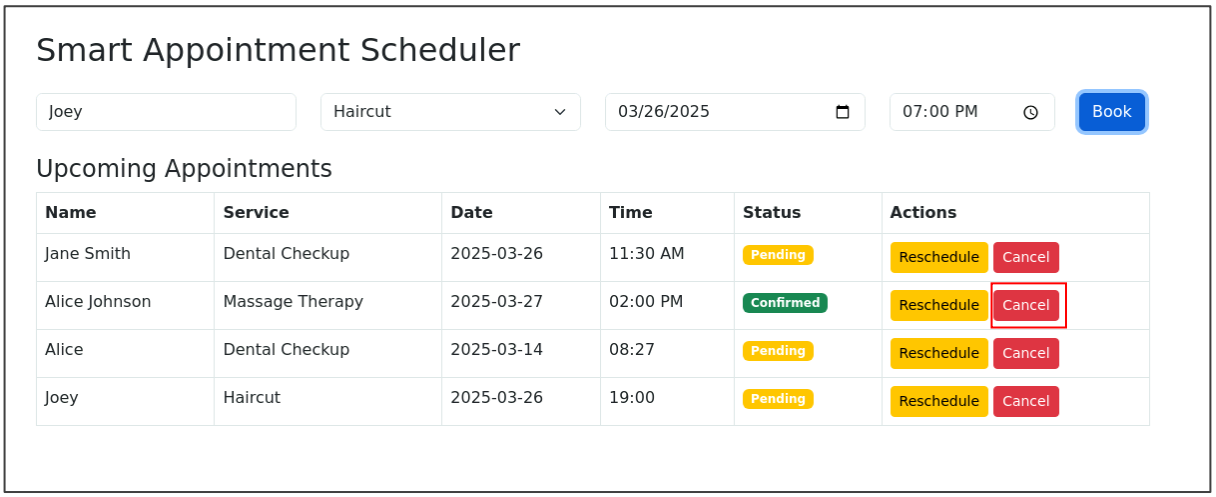
5.2 Provide your name, select a service, date, and time to **Book** an appointment using the form



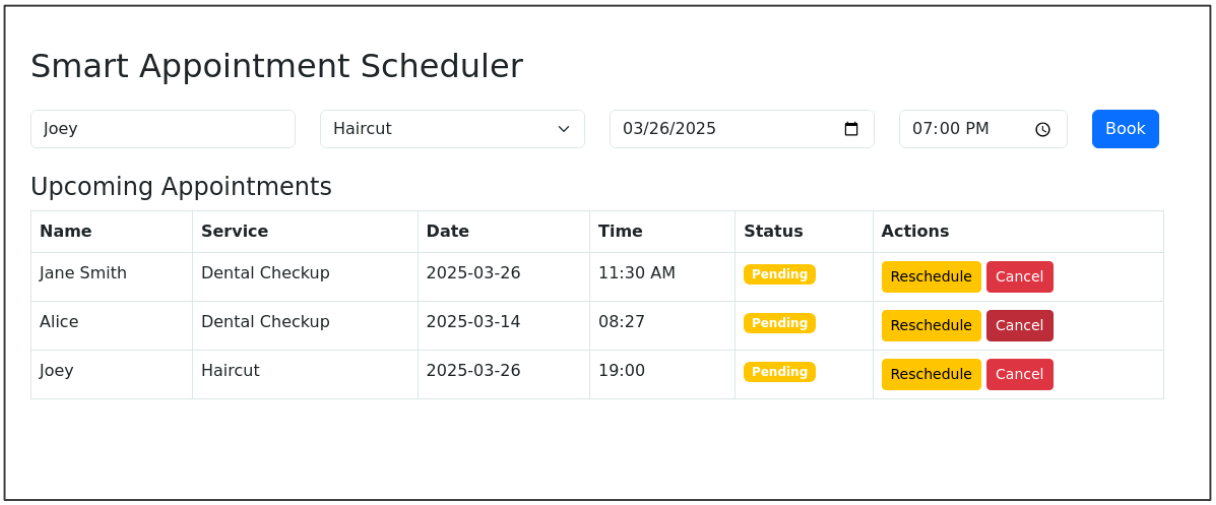
The appointment is booked successfully as shown below:



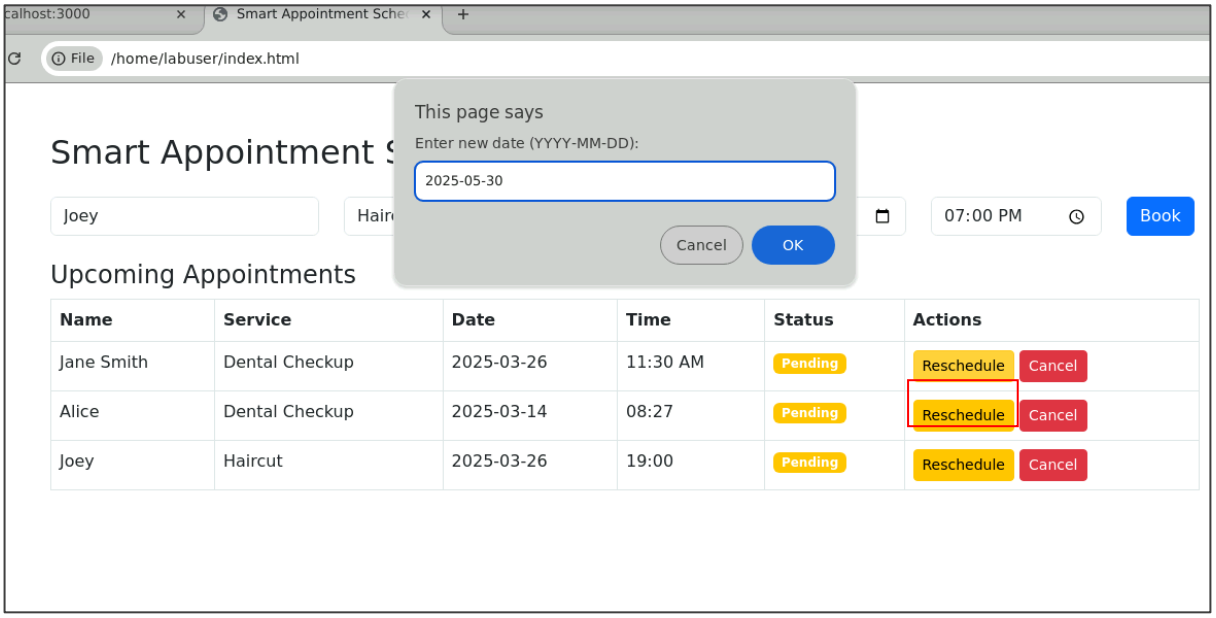
5.3 Click the **Cancel** button next to an appointment to remove it from the list

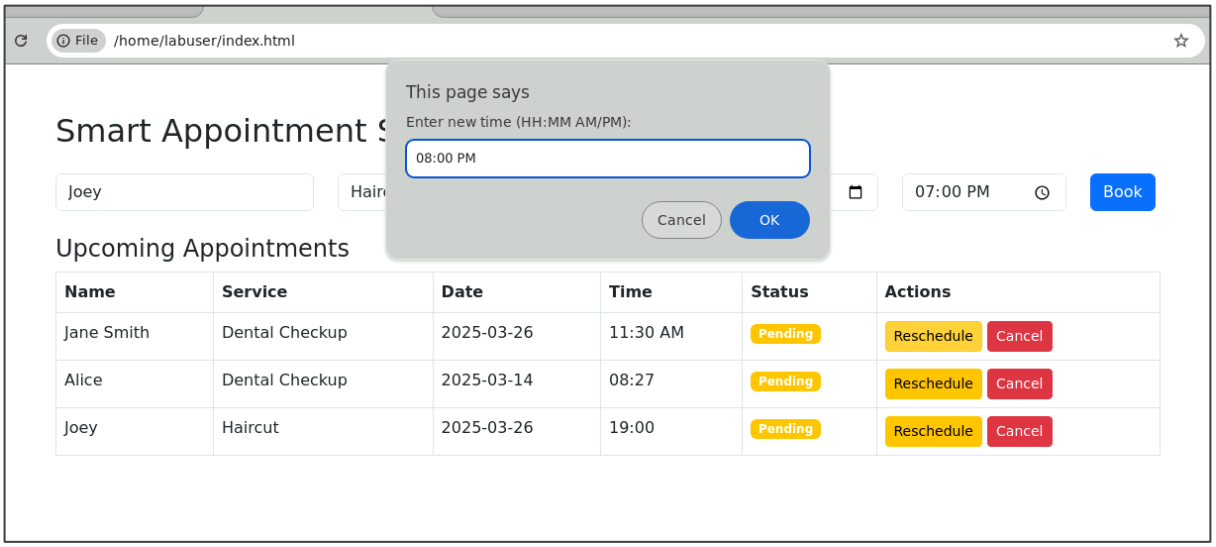


Upon cancellation, the appointment no longer appears in the list as shown below:

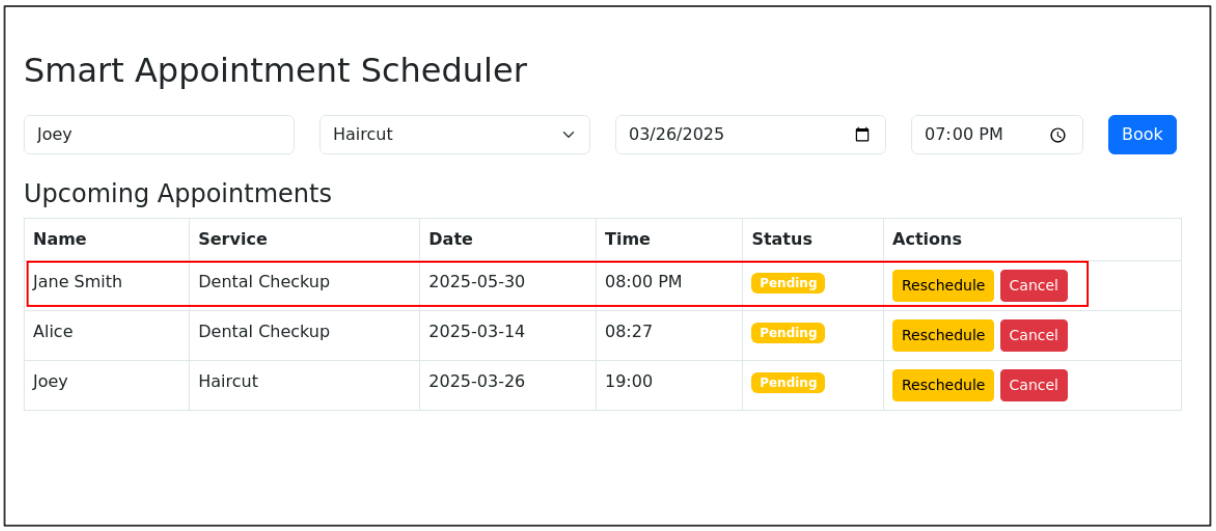


5.4 Click the **Reschedule** button and update the date and time to modify the appointment





The updated date and time are reflected in the appointment list as shown below:



By following the above steps, you have successfully built and tested a real-time Smart Appointment Scheduler application using HTML, CSS, JavaScript, and JSON Server. This includes setting up the mock backend, designing a responsive user interface, implementing dynamic appointment logic, and verifying real-time updates. These steps ensure a functional, interactive system that enhances appointment management and demonstrates practical integration of frontend and backend development skills.